

AN ARDUINO BASED BRAILLE DISPLAY FOR THE VISUALLY IMPAIRED

A Project report submitted in partial fulfilment of the requirement for the degree of the

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

SUBMITTED BY

SARAGADAM PRUDHVI TEJA (Regd. No.: 319106512045)

CHAGANTIPATI TARUN (Regd. No.: 319106512007)

SANJAY RAMASWAMY GOTHIREDDY (Regd. No.: 319106512044)

Under the esteemed guidance of

Prof. P.V. SRIDEVI, M.E., Ph.D.



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING,

ANDHRA UNIVERSITY COLLEGE OF ENGINEERING,

ANDHRA UNIVERSITY,

Visakhapatnam-530003,

2022-2023

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING,
ANDHRA UNIVERSITY COLLEGE OF ENGINEERING,
ANDHRA UNIVERSITY,VISAKHAPATNAM-530003,
2022-2023**



CERTIFICATE

This is to certify that the project report entitled, “**AN ARDUINO BASED BRAILLE DISPLAY FOR THE VISUALLY IMPAIRED**”, is a bonafide project by **SARAGADAM PRUDHVI TEJA**(Regd. No.: 319106512045), **CHAGANTIPATI TARUN**(Regd. No.: 319106512007), **SANJAY RAMASWAMY GOTHIREDDY**(Regd. No.: 319106512044), during the year **2022-2023** for the partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the department of **ELECTRONICS AND COMMUNICATION ENGINEERING**, Andhra University College of Engineering, Andhra University, Visakhapatnam.

Project Guide

PROF.P.V. SRIDEVI,M.E., Ph.D.

Dept. of Electronics &
Communication Engineering,
Andhra University College of Engineering,
Andhra University,
Visakhapatnam - 530003.

Head of the Department

PROF.P. RAJESH KUMAR,M.E., Ph.D.

Dept. of Electronics &
Communication Engineering,
Andhra University College of Engineering,
Andhra University,
Visakhapatnam - 530003.

DECLARATION

We, SARAGADAM PRUDHVI TEJA (Regd. No.: 319106512045), CHAGANTIPATI TARUN (Regd. No.: 319106512007), SANJAY RAMASWAMY GOTHIREDDY (Regd. No.: 319106512044), hereby declare that the project entitled **“AN ARDUINO BASED BRAILLE DISPLAY FOR THE VISUALLY IMPAIRED”** is an original work done at ANDHRA UNIVERSITY COLLEGE OF ENGINEERING, VISAKHAPATNAM, submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING. We assure that this project is not submitted in any university or college.

Place: Visakhapatnam

SARAGADAM PRUDHVI TEJA (Reg. No.: 319106512045)

Date:

CHAGANTIPATI TARUN (Reg. No.: 319106512007)

SANJAY RAMASWAMY GOTHIREDDY (Reg. No.: 319106512044)

ACKNOWLEDGEMENT

It is with great sense of satisfaction that we present “**AN ARDUINO BASED BRAILLE DISPLAY FOR THE VISUALLY IMPAIRED**” in the form of final project.

We express our deep sense of gratitude to our project guide, **Prof.P.V.SRIDEVI**, Department of Electronics and Communication Engineering, Andhra University College of Engineering, for guiding us all through the project work, giving right direction and shape to our learning by extending her expertise and experience in the study. We are deeply indebted for her excellent and enlightened guidance.

We are very thankful to our beloved Head of the Department, **Prof.P.Rajesh Kumar**, Department of Electronics and Communication Engineering, Andhra University College of Engineering, for his valuable suggestions and constant motivation that greatly helped the project to successfully complete.

We express our sincere gratitude to Hon’ble Vice-Chancellor **Prof.P.V.G.D.Prasad Reddy**, Andhra University, **Prof.G.Sasi Bhushan Rao**, Principal, A.U. College of Engineering, **Prof.M.S.Anuradha**, **Dr.G.Rajeshwara Rao**, Adjunct professor, **Dr.V.Malleswara Rao**, Adjunct professor, **Dr.S.Aruna**, Assistant professor, Department of Electronics and Communication Engineering, Andhra University College of Engineering, Andhra University, Visakhapatnam for their kind encouragement.

We also extend our heartfelt gratitude to all the non-teaching staff of the Department of Electronics and Communication Engineering for their support.

Finally, we express our indebtedness to our beloved parents and friends without whose blessing and encouragement we would not have completed our work fruitfully.

Regards

SARAGADAM PRUDHVI TEJA (Regd. No.: 319106512045)

CHAGANTIPATI TARUN (Regd. No.: 319106512007)

SANJAY RAMASWAMY GOTHIREDDY (Regd. No.: 319106512044)

ABSTRACT

The aim of this project is to design and develop an affordable and portable Braille tutor device for visually impaired learners. Braille is a tactile writing system that is widely used by people who are blind or have low vision. However, learning braille can be challenging, especially for those who do not have access to specialized training and equipment.

This project uses a microcontroller (Arduino UNO) and an Arduino IDE software to interpret and process the stored input text. The microcontroller is responsible for controlling the input and output devices by executing the program instructions. A translation algorithm uploaded in Arduino IDE which converts the input text which is stored in the SD card into braille.

The Arduino sends the signal to the respective relays which is needed to perform the switching action. The powering for Arduino is given by the USB cable connected to laptop, and for relays by LM2596 buck converter, which is powered by an external battery .While switching action of relay, the push-pull solenoids interfaced to respective relays pops up and creates a refreshable braille display.

Now a days as technology is increasing, from books to smartphones (i.e., Kindle, tabs, etc).But for the case of visually impaired, it should be from braille books to refreshable braille displays, that are designed in this project.

INDEX

Acknowledgement

Abstract

Chapter 1: INTRODUCTION	1
1.1 Introduction to Braille System	1
1.2 Various Braille Displays	2
1.2.1 Grade 1 Braille	2
1.2.2 Grade 2 Braille	2
1.2.3 Grade 3 Braille Cell	3
1.3 A Closer Analysis of Braille	4
Chapter 2: LITERATURE REVIEW	8
Chapter 3: DESIGN METHODOLOGY	10
3.1 Components Used	10
3.2 Description and Working of each Component	11
3.2.1 Arduino UNO	11
3.2.2 5V 4-Channel Relay	12
3.2.3 LM2596 Buck Converter Module	14
3.2.4 Push-pull Solenoids	16
3.2.5 SD Card Module	18
3.2.6 Other Components	21
3.3 Circuit Diagram	22
3.4 Look-up Table	24
Chapter 4: IMPLEMENTATION	25
4.1 Flowchart	25
4.2 Uploading Text	26
4.3 SD card Initialisation	26
4.4 Recognition and Translation	28
4.5 Switching action of Relay	28
4.6 Triggering Solenoids	29

Chapter 5: RESULTS AND DISCUSSION	31
5.1 Output for Example Text 1	31
5.2 Output for Example Text 2	31
5.3 Output for Example Text 3	32
5.4 Output for Example Text 4	32
5.5 Applications	33
5.6 Advantages and Disadvantages	33
CONCLUSION AND FUTURE SCOPE	34
BIBLIOGRAPHY	35
APPENDIX	37
I. Code	37
II. Specification sheets	53

LIST OF FIGURES

Fig 1.1 : Grade 1 braille system	2
Fig 1.2 : Grade 2 braille system	3
Fig 1.3 : Single braille cell	4
Fig 1.4 : Literary and Nemeth braille representation	5
Fig 3.1 : Arduino UNO ATmega238p	11
Fig 3.2 : 5V 4-channel relay module	13
Fig 3.3 : Relay interfacing with Arduino	14
Fig 3.4 : LM2596 Buck converter module	15
Fig 3.5 : Internal structure of LM2596 Buck converter module	16
Fig 3.6 : Push-pull solenoid	17
Fig 3.7 : Internal structure of a solenoid	17
Fig 3.8 : Solenoid in resting phase	18
Fig 3.9 : Solenoid in active phase	18
Fig 3.10 : SD card module	19
Fig 3.11 : Interfacing SD card module with Arduino	20
Fig 3.12 : Other components of an Arduino based braille display for the visually impaired	21
Fig 3.13 : Circuit diagram of an Arduino based braille display for the visually impaired	22
Fig 4.1 : Flowchart of an Arduino based braille display for visually impaired	25
Fig 4.2 : Uploading text files	26
Fig 4.3 : SD card initialization	27
Fig 4.4 : Relay switching mechanism	29
Fig 4.5 : Braille cell using solenoids triggering	29
Fig 5.1 : Outputs of an Arduino based braille display for the visually impaired	32

LIST OF TABLES

Table 3.1 - List of components	10
Table 3.2 - Braille mapping of letters into solenoid display	24
Table 3.3 - Braille mapping of all other characters into solenoid display	24

LIST OF ABBREVIATIONS

ATMEL	-	Advanced Technology for Memory and Logic
AVR	-	Automatic voltage regulator
ATMEGA	-	ATMEL Mega AVR
ARM	-	Advanced Risc Machine
IDE	-	Integrated Development Environment
UNO	-	one in Italian, 1 st edition
LED	-	Light Emitting Diode
SMPS	-	Switched Mode Power Supply
RPS	-	Regulated Power Supply
MOSFET	-	Metal Oxide Semiconductor Field Effect Transistor
SD	-	Secure Digital
MISO	-	Master Input Slave Output
MOSI	-	Master Output Slave Input
SPI	-	Serial Peripheral Interface
SCK	-	Serial Clock
CS	-	Chip Select
FAT16	-	File Allocation Table,16bit file system
FAT32	-	File Allocation Table,32bit file system
BPS	-	Bits per system
NO	-	Normally Open
NC	-	Normally Closed

Chapter 1: INTRODUCTION

This chapter discusses about Braille representation system and the various existing Braille displays, solenoid based refreshable braille display, Literary and Nemeth braille representations and programming language used .

1.1 Introduction to Braille System

Braille is a tactile writing system used by people who are blind, deafblind or have low vision. It can be read either on embossed paper or by using refreshable braille displays . Braille is a code, not a language. It is a system of reading and writing a specific language without the use of sight. Braille enables people with blindness and visual impairments to read through touch.

Though Louis Braille created the tactile reading and writing system that is in use today, he drew inspiration from a French army captain named Charles Barbier who developed a similar code. Barbier created a nonverbal communication system so his officers could read battle commands during the night without the use of candlelight. In the late 1800s, Louis Braille refined Barbier's system and created the foundation for today's braille code.

Braille characters are formed using a combination of six raised dots arranged in a 3×2 matrix, called the braille cell. The number and arrangement of these dots distinguishes one character from another. Since the various braille alphabets originated as transcription codes for printed writing, the mappings (sets of character designations) vary from language to language, and even within one; in English Braille there are 3 levels of braille: uncontracted braille – a letter-by-letter transcription used for basic literacy; contracted braille – an addition of abbreviations and contractions used as a space-saving mechanism; and grade 3 – various non-standardized personal stenography that is less commonly used.

In addition to braille text (letters, punctuation, contractions), it is also possible to create embossed illustrations and graphs, with the lines either solid or made of series of dots, arrows, and bullets that are larger than braille dots. A full braille cell includes six raised dots arranged in two columns, each column having three dots. The dot positions are identified by numbers from one to six. There are 64 possible combinations, including no dots at all for a word space. Dot configurations can be used to represent a letter, digit, punctuation mark, or even a word.

1.2 Various Braille Displays

1.2.1 Grade 1 Braille

Grade 1 braille is a letter-for-letter substitution of its printed counterpart. This is the preferred code for beginners because it allows people to get familiar with, and recognize different aspects of, the code while learning how to read braille.

English grade 1 braille consists of the 26 standard letters of the alphabet as well as punctuation.

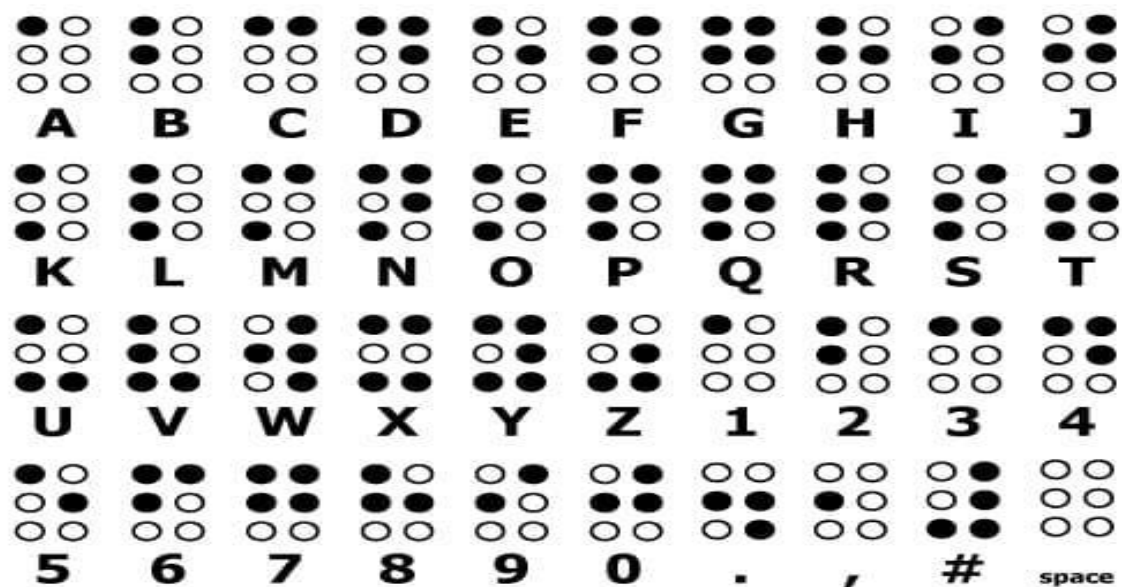


Fig 1.1: Grade 1 braille system

1.2.2 Grade 2 Braille

The literary braille code, grade 2, uses “contractions” that substitute shorter sequences for the full spelling of commonly-occurring letter groups. The contractions are similar to English print contractions, like “cannot” versus “can’t”, in the way that a word is shortened. For example, “the” is usually a single character in braille.

A standard braille cell is large. That means that one page of print can easily turn into three pages of braille. Contractions help reduce the number of characters and thereby reduce the overall size of a document. Second, reading and writing braille can be time-consuming. By implementing contractions, it takes less time to do both.

Grade 2 braille is the most commonly used form of braille code and is found in books, public signage, and restaurant menus to name a few. It consists of the 26 standard letters of the alphabet, punctuation, and contractions.

	but	can	do	every	from	go	have		just
knowledge	like	more	not		people	quite	rather	so	that
us	very	it	you	as	wil				

and	ar	by	cc	ch	com	dd	ea	ed	en
		was	con	child		dis			enough
er	ff	for	gg	gh	in	ing	of	ou	ow
	to		were					out	
sh	st	the	th	wh	with				
shall	still		this	which					

Fig.1.2: Grade 2 braille display

1.2.3 Grade 3 Braille Cell

Grade 3 is the last, and certainly least used form of literary braille code used within the blind community. Considered braille “shorthand,” this code often compresses entire words into just one or a few characters. There is no official standardization for grade 3 braille and, therefore, official publications do not use it. Instead, it most often appears in personal letters, diaries, and notes.

1.3 A Closer Analysis of Braille

Six Dots

Six dot positions make up each braille character or “cell”. These dot positions form a rectangle composed of 2 columns with 3 dots in each column. A single dot or any combination of dots may be raised at any of the 6 positions. Counting spaces, in which no dots appear, there are 64 (2^6) English braille combinations in total. When referencing a braille character, one may describe the positions where dots are raised. Each dot within a cell has a number. Starting in the upper left and moving down, the dots are universally numbered 1 through 6 as seen in the image below.

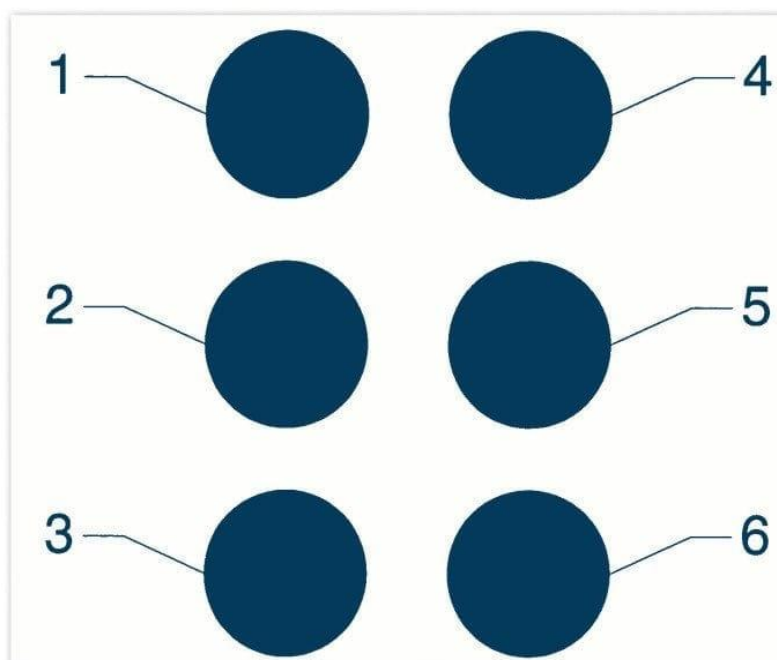


Fig 1.3: Single braille cell

For example, dots 1-3-4 would describe a cell with three dots raised; the top and bottom cells of the left column and the top cell of the right column.

Because the 64 distinct characters are never enough to cover all possible print signs and their variations, it is necessary to use multi-character sequences for some purposes. Often, it uses certain characters as “prefixes” or “indicators” that affect the meaning of subsequent cells. For example, when a dot 6 falls before a letter, the reader knows it is a capital letter. Without a dot 6, the reader knows it is a lower-case letter. In another example, dots 3-4-5-6, called the

“numeric indicator”, tells the reader to interpret certain following letters (A through J) as numbers. There is then another character or a space to tell the reader they are switching back to letters.

Using Nematic Braille Code to solve the number-character ambiguity

In general, the first 10 alphabets and the numbers from 0-9 have same braille format in Literary format. So, to clear this misconception Nemeth Code.

NUMBERS											
											Literary Code
#	0	1	2	3	4	5	6	7	8	9	
											Nemeth Code
#	0	1	2	3	4	5	6	7	8	9	

Fig 1.4: Literary and Nemeth braille representation

The Nemeth code clears the air from the problem of misunderstanding between alphabets and numbers.

SOFTWARE

Arduino IDE is used to develop code for Arduino boards. It is a software application that allows users to write, compile, and upload code to Arduino boards. The Arduino IDE is available for Windows, Mac OS X, and Linux operating systems [10].

The IDE comes with a simple code editor, a serial monitor, and a compiler. It also has a library manager to manage third-party libraries and examples. The code editor includes features like syntax highlighting, auto-complete, and error highlighting. The serial monitor allows users to communicate with the Arduino board and view data transmitted over serial communication.



Fig 1.5: Arduino IDE window

Arduino IDE supports the C and C++ programming languages, with a specific set of libraries and functions tailored for Arduino boards. The IDE simplifies the programming process for beginners and provides a streamlined development process for advanced users. Additionally, Arduino IDE is open-source software, which means users can contribute to its development and customize it for their own use.

MOTIVATION

The motivation for an Arduino based Braille display for the visually impaired project is to make information more accessible to visually impaired individuals. With this project, most of the visually challenged individuals can be empowered by providing them with an efficient and effective way to read and access written content. This project can improve their quality of life by giving them access to more information and helping them become more independent in their daily lives. Additionally, this project can promote inclusivity and equality by bridging the gap between sighted and visually challenged individuals.

OBJECTIVE

The objective of the "An Arduino based braille display for the visually impaired" project is to create a system that can convert digital text into braille code, which can then be read by visually challenged individuals using a braille display or similar tactile interface. The system should aim to be user-friendly and affordable, and should provide an efficient and accurate means of converting text into braille. Additionally, the project aims to promote accessibility and inclusivity for visually challenged individuals in various aspects, such as education, employment, and daily life.

Chapter 2: LITERATURE REVIEW

First, in order to design a hardware model for text to braille conversion accurately and efficiently & before providing our own solution in depth, one must initially understand the existing research and work done in the field. Therefore, in this section many different research papers were investigated to collect relevant information about the project.

Ghose R, *et.al.*, [1] has done an efficient work in “*Architecture of a web browser for visually handicapped people*”, which was only a web platform for visually challenged but it served as a motivation to implement a physical platform for visually challenged to interact and learn.

P. H. Zope, *et.al.*, [2] has contributed in “*Design and Implementation of Messaging System Using Braille Code for Virtually Impaired Persons*”, which presents a messaging system that uses braille code to facilitate communication for visually impaired individuals.

P. J. Swati Malik, [3] published a paper titled “*Low-Cost Portable E-Braille Device for Blind and Visually Impaired Persons*”, The device uses an Arduino microcontroller and a refreshable Braille display to provide a tactile output for users, but it mainly lacks the ability to read and display the stored data in text files.

Priyanka D. Joshi, *et.al.*, [4] has done extensive “*Microcontroller Based Smart Touch Gadget Assistant for Visually Impaired*”, The device uses touch sensors and voice recognition to provide assistance to the user. The voice recognition system is used for controlling the device and interacting with the user. The main disadvantage of this idea is not cost-effective approach and it is not effective in recognizing voice when there is high background noise.

S. B. Sibila.R, *et.al.*, [5] has contributed to the software analysis of “*Text to Braille Conversion Using MATLAB*”, Which is bounded only to software version that needs to be converted into hardware design in order to make use of the idea.

S. Bharath Reddy, *et.al.*, [6] has contributed for “*Design of a Tactile Braille Cell*”, which does include the concept of using a braille cell designed using 3D printing technology and consists of a solenoid actuator, a brass pin, and a return spring, but does not include advanced features like stored text file reading and displaying simultaneously.

A V Hemanth, *et.al.*, [7] has contributed in the development of “*Arduino and Solenoid Valve-Based Text to Braille Converter*”, but this project is only limited to alphabetic inputs and also output of numbers and alphabets coincide so it may be difficult for user to differentiate.

Rhea Sawant, *et.al.*, [8] has extensively worked and contributed on “*Text to Braille Conversion System*”, gives an insights of the design and implementation of a text-to-braille conversion system which aims to finally provide them with a means of accessing digital content.

Numerous papers were proposed with various approaches to convert text to braille display based on methods both software & hardware, but this approach includes features like text extraction, character recognition & translation, respective solenoid switching for creating refreshable braille impressions of every character.

Problems with above approaches is they need expert for feature extraction, continuous monitoring, low accuracy and cannot be automated. On contrary to above methods our model is based on electronic approach & a physical hardware solution with additional simultaneous text reading and display feature which makes it easy to automate and mass produce at a cheaper cost.

The proposed approach has the potential for achieving very high classification accuracy, saving tremendous time for data learning. The proposed solution employs a novel architecture consisting of multiple blocks of integrated components.

Chapter 3: DESIGN METHODOLOGY

This chapters deals with the requirement of components for the implementation of this project, and also the detailed description and working of each and every component used.

3.1 Components Used

The following Table 3.1 shows the list of all components used in the hardware design of this device.

Sl.	Name of Component	Manufacturer and Part Number	Qty	Description
1.	Microcontroller	Arduino Uno	1	9V-42mA (ATmega328p)
2.	5V 4-Channel Relay	Xcluma, BE-000035	2	5V, 70mA trigger current
3.	LM2596 Buck	IC Transistors	1	9V to 5V convertor
4.	Push-Pull Solenoids	JF-0630	6	12V,300mA,6N,10mm
5.	SD Card Module	Generic, Micro SDHC	1	5V operating Voltage
6.	SD Card	microSD	1	4GB
7.	Jumper Wires	Sparkfun, PRT-12794	15	Male-Male, Male-Female
8.	Half Breadboard	Quartz Comp,QC0441	1	400 Point ½ size solderless
9.	Arduino Cable	Arduino	1	Arduino Powering
10.	Barrel-Jumper plug	generic	3	Pos, Neg (GND), Sleeve
11.	Charging Adaptor	Mass Power	3	12V,1A Power Adaptor

Table 3.1- List of components

3.2 Description and Working of each Component

3.2.1. ARDUINO UNO

The Arduino is a board based on an ATMEL AVR microcontroller. Microcontrollers are integrated circuits where instructions can be recorded, which user write with the programming language that can be used in the Arduino IDE environment. These instructions allow user to create programs that interact with the circuitry on the board.

The most commonly used microcontrollers on Arduino platforms are the Atmega168, Atmega328, Atmega1280, ATmega8 for their simplicity, but it is being expanded to ATMEL microcontrollers with 32-bit ARM architecture and also to Intel microcontrollers.

The Arduino microcontroller has communication ports and input/output ports. with which can connect different types of peripherals on the board. The information of these peripherals that you connect will be transferred to the microcontroller, On the other hand, Arduino provides the user with software consisting of an IDE , the tools to transfer the firmware to the microcontroller and the bootloader executed on the board. Arduino is an open-source electronics creation platform, which is based on free, flexible, and easy to use hardware and software for creators and developers [10]. This platform allows user to create different types of single-board microcomputers to which the community of creators can give different types of use.

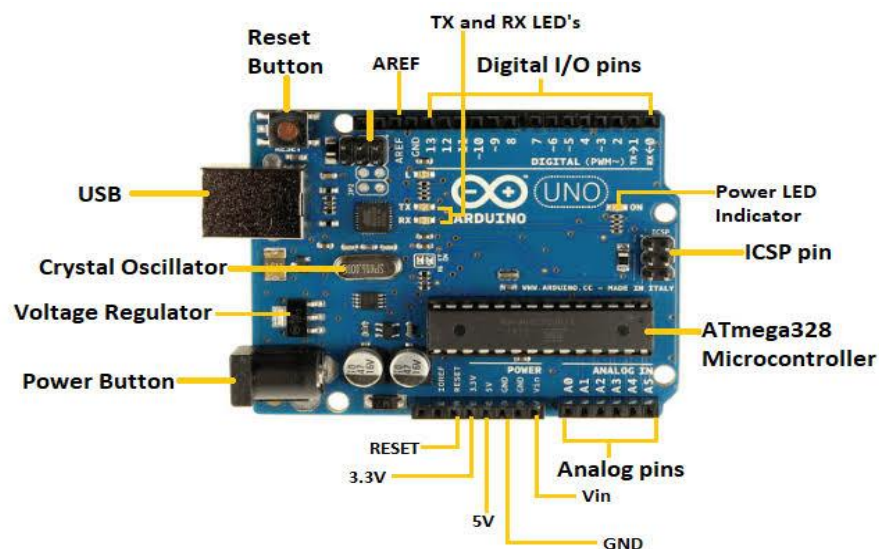


Fig 3.1: Arduino UNO ATmega328p

Arduino simplifies the work with microcontrollers and offers the following advantages: cheap, multiplatform, simple programming environment, free and extensible software through C ++ libraries, free and extensible hardware.

When working with Arduino, different technology concepts are handled with priori that have nothing to do with each other but unify them: digital and analog electronics, electricity, programming, microcontrollers, signal processing, communication protocols, processor architecture, mechanics, motors, electronic board design and so on.

3.2.2. 5V 4-CHANNEL RELAY MODULE

A 5V relay module is a single or multi-channel relay module that works with a low-level trigger voltage of 5V DC. The input voltage can be from any microcontroller or logic chip that outputs a digital signal.

Like most other relays, the 5V relay module is an electrically operated, electromagnetic switch that can be used to turn ON or turn OFF a circuit. It consists of two parts: the relay itself and the control module.

- The relay contains the coil that creates the magnetic field, the armature that move to complete or disconnect a circuit, and contacts that open and close to operate the load switch.
- The relay control module is the interface or part of the relay module that the user interacts. It contains the input terminals for connecting to the microcontroller, as well as the output terminals for connecting to the load.
- The control module also contains LED indicators for power and status and other devices such as protection diode, transistor, resistor, and other semiconductor devices necessary for its operation.

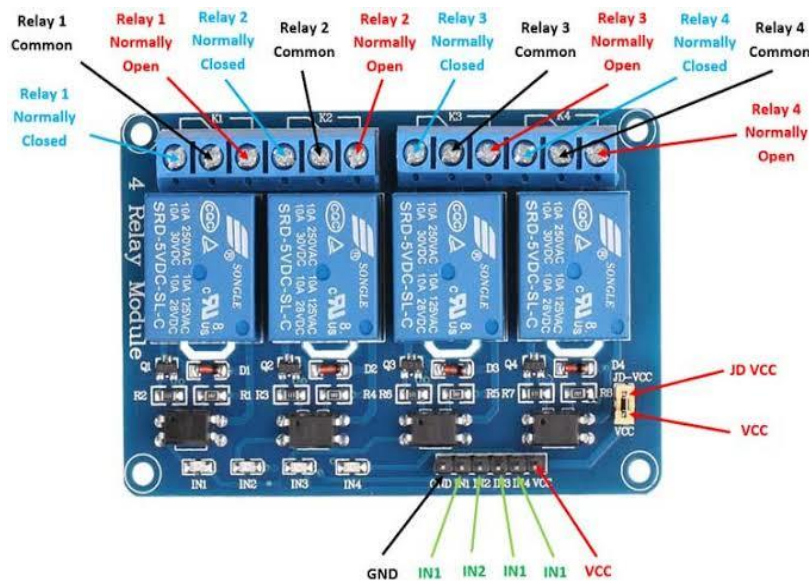


Fig 3.2: 5V 4-Channel relay module

Working of a Relay Module

The 5V relay module requires a 5V signal delivered from a microcontroller or sensor to trigger the switch. Its working is also very simple. When the input pin is HIGH, the relay turns ON, and when the input pin is LOW it turns OFF. Below is the 5V relay module working principle.

- The relay is activated by a low-level trigger signal applied to its IN1 or IN2 pin. When the trigger signal is applied, the transistor turns ON and amplifies the signal.
- This triggers the relay to turn ON and connect the load to either the NO or NC pin. The LED will light up to indicate that the relay is ON.
- When the trigger signal is removed, the transistor turns OFF and the relay turns OFF. The load is then disconnected from the NO or NC pin. The LED will turn OFF to indicate that the relay is OFF.

The 5V relay module can be used to control a load such as a lighting system, motor, or solenoid. It can also be used to switch AC or DC voltages. The maximum voltage and current that the 5V relay module can control is dependent on the specifications of the relay.

Interfacing Relay Module with Arduino

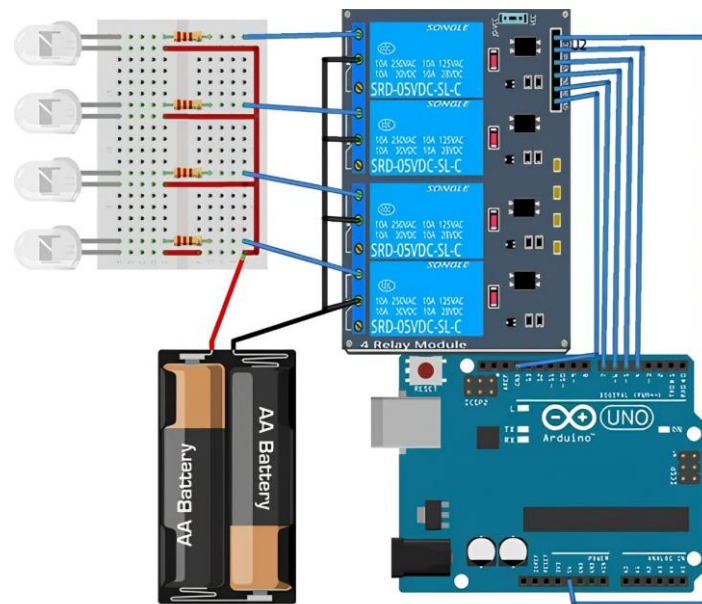


Fig 3.3: Relay interfacing with Arduino

The 5V relay module can be easily interfaced with an Arduino [11]. The above diagram shows how to connect a single-channel relay module with Arduino microcontroller. The 5V relay module Arduino wiring is very simple:

The VCC and GND pins of the relay module are connected to the 5V and GND pins of the Arduino, IN1 pin of the relay module is connected to a digital output pin of the Arduino, and the load is connected to the NO and COM pins of the relay module.

- When the digital output pin is set to HIGH, the module turns ON and activates the relay. This will turn ON the load connected to the NO and COM pins of the relay module.
- When the digital output pin is set to LOW, the circuit OFF and deactivates the relay, turning OFF the load.

3.2.3 LM2596 BUCK CONVERTER MODULE

The LM2596 is voltage-regulating module that can output voltage values ranging from 1.25V to 35V by adjusting potentiometer with ease. It is known for its high current rating of 3A and is available in many versions with fixed output voltage like 3.3V, 5V and 12V. But, the most

famous one is the LM2596-ADJ which has variable output voltage. The Integrated circuit is basically a buck converter that operates on 150KHz switching frequency, it takes in an input voltage and uses the internal potentiometer to regulated a desired output voltage. It has high efficiency and in-built thermal shutdown and current limit functionalities.

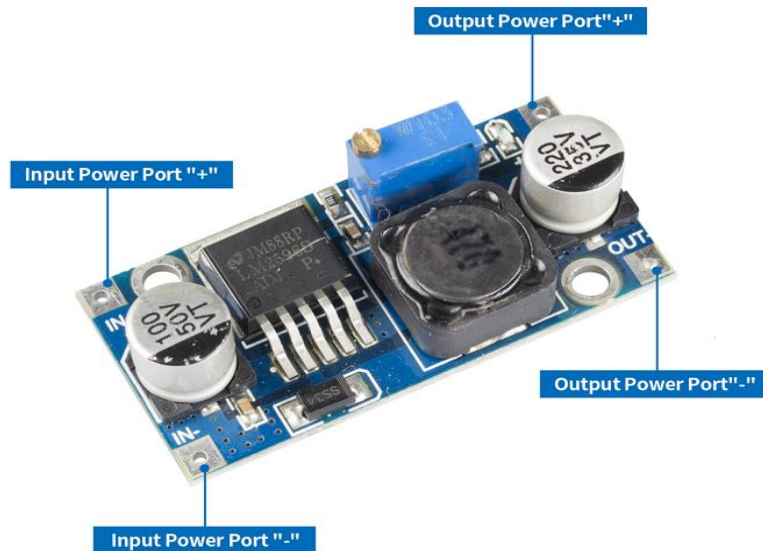


Fig 3.4: LM2596 Buck converter module

Working Principle

1. Input voltage range: DC 3.2V~40V (the input voltage must be more than 1.5V higher than the voltage to be output.)
2. Output voltage range: DC 1.25V~35V continuous adjustable voltage with high efficiency (up to 92%); the output current can reach 3A. Get a power supply (3-40V), the power indicator light is on and the module is working well.
3. Adjust the blue potentiometer knob (generally clockwise rotation makes boost, and turning counter-clockwise makes step-down). Monitor the output voltage to get the desired voltage with a multimeter.

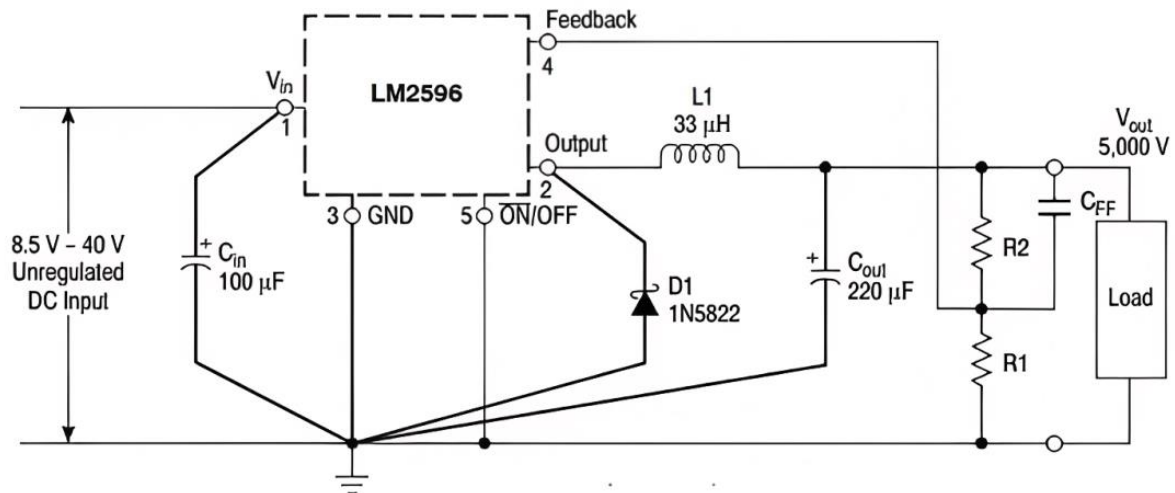


Fig 3.5: Internal structure of LM2596 Buck converter module

Applications of LM2596 Buck converter

- Used in battery circuits since they have high efficiency
- Step-down linear regulators
- Used in small SMPS circuits
- Battery operated applications
- Variable voltage generators
- Miniature RPS circuits

3.2.4. PUSH-PULL SOLENOIDS

A solenoid is a long piece of wire which is wound in the shape of a coil. When the electric current passes through the coil it creates a relatively uniform magnetic field inside the coil. Two basic laws govern the functioning of a solenoid: Faraday's Law and Ampere's Law [9]. This solenoid comes under linear solenoids and its functioning is analogous to relays.



Fig 3.6: Push-pull solenoid

Solenoids have been divided into two main categories:

1. Linear solenoids
2. Rotary solenoids

Working of a Push-pull Solenoid

The solenoid simply works on the principle of Electromagnetism. When the current flow through the coil magnetic field is generated in it, when a metal core is placed inside the coil, the magnetic lines of flux is concentrated on the core which increases the induction of the coil as compared to the air core.

This project solely requires the linear version of push-pull solenoids as the linear motion is helpful for creating a pop-up braille impression.

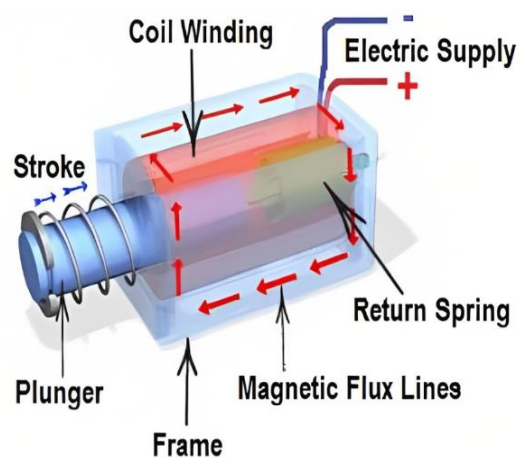


Fig 3.7: Internal structure of a solenoid

Linear solenoid is an electromagnetic or electromechanical device, which convert electrical power into a magnetic signal or mechanical motion. It works on the same principle as the electromechanical relays and can be controlled by using MOSFET, Transistors, etc. It can be designed for proportional motion relative to power input, includes push and pulls application. They are used to provide high force, because they are developed by the combination of high magnetic flux and ferromagnetic material to provide more power.

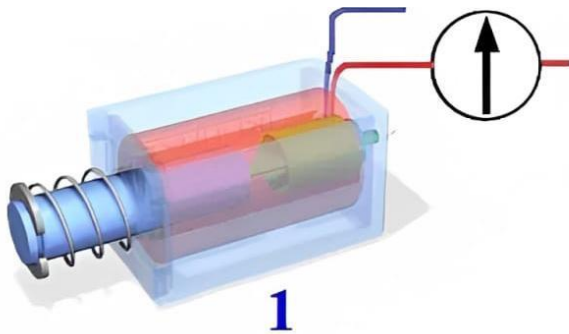


Fig 3.8: Solenoid in resting phase

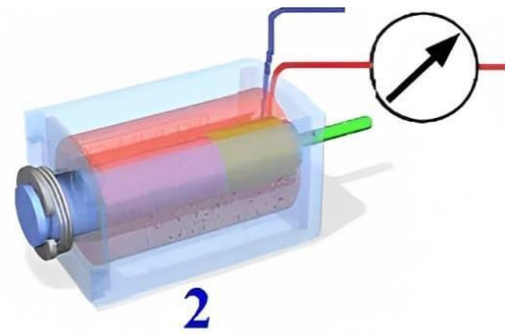


Fig 3.9: Solenoid is in active phase

The solenoid is pushed away when the power supply is ON as there is a significant current flowing the solenoids & the solenoids are retreated back whenever the power supply is OFF as there won't be enough current, so the solenoids get pulled back to its original position. The solenoids are hold still as long as there is a sufficient constant DC current flowing through it.

3.2.5. SD CARD MODULE

An SD card module or a Micro SD card adapter is a simple board which facilitates connection between a Micro SD card and a microcontroller like Arduino [12]. The following is the image of a typical SD card module.

SD card module can be used in any project that requires data reading and writing. The board has built in 3.3V voltage regulator. A standard SD card, in which data is usually logged, can be directly inserted into the Arduino board in order to read/write operations.

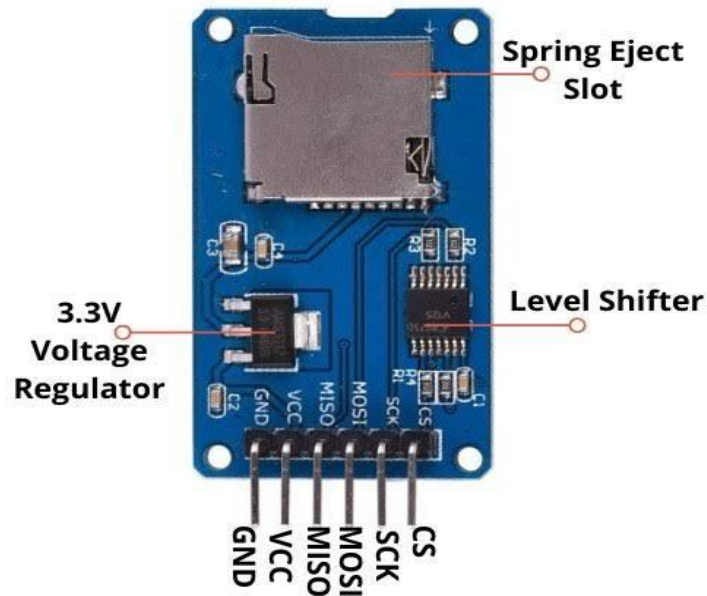


Fig 3.10: SD card module

Pin description of SD card module

- VCC pin provides power to the module and should be connected to the Arduino's 5V pin.
- GND is a ground pin.
- MISO is the SPI output from the microSD card module.
- MOSI is the SPI input to the microSD card module.
- SCK pin accepts clock pulses from the master (an Arduino in our case) to synchronize data transmission.
- CS pin is a control pin that is used to select one or a set of slave devices on the SPI bus.

The sketch begins by including the SD library and the SPI library in the Arduino code, which allow us to communicate with the SD card via the SPI interface

1. `#include <SPI.h>`

2. `#include<SD.h>`

Interfacing SD card module with Arduino

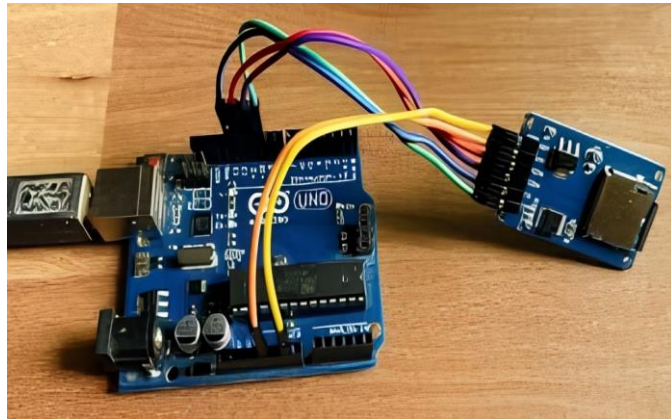
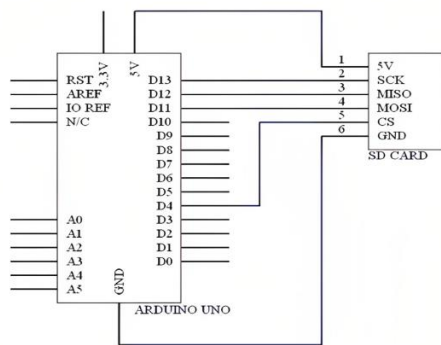


Fig 3.11: Interfacing SD card module with Arduino

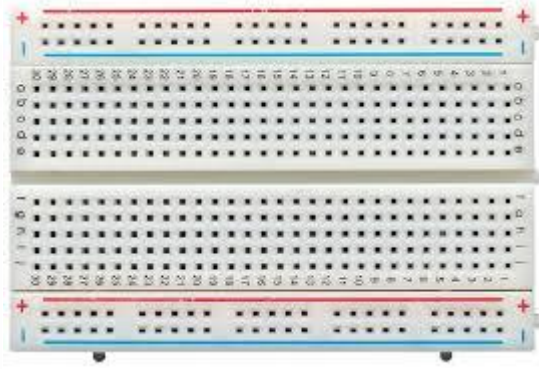
The connections between the Arduino & SD card module are as follows

1. 5V pin of SD card module is connected to 5V voltage regulator output pin of Arduino.
2. SCK pin is connected to D13 (Digital Pin 13) of Arduino.
3. MISO pin is connected to D12 (Digital Pin 12) of Arduino.
4. MOSI pin is connected to D11 (Digital Pin 11) of Arduino.
5. CS pin is connected to D4 (Digital Pin 4) of Arduino.

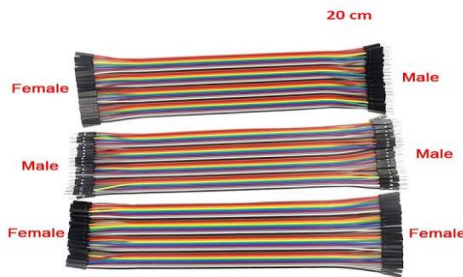
3.2.6. OTHER COMPONENTS



USB CABLE



HALF BREADBOARD



JUMPER WIRES



MICRO SD CARD



JUMPER-BARREL CONNECTOR



POWER ADAPTORS

Fig 3.12: Other components of an Arduino based braille display for the visually impaired

3.3. CIRCUIT DIAGRAM

The following Fig 3.3 shows the detailed circuit description of an Arduino based braille display for visually impaired.

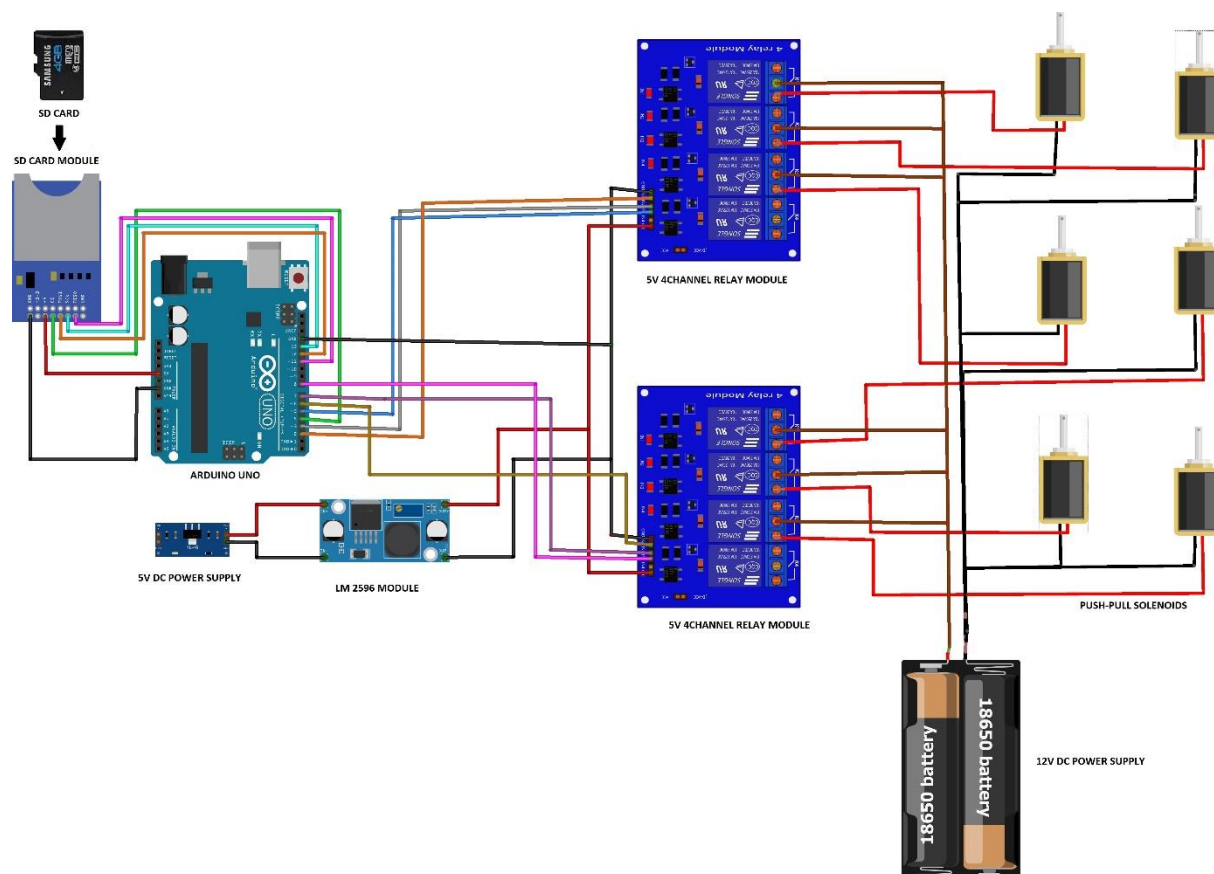


Fig 3.13: Circuit diagram of an Arduino based braille display for the visually impaired

The circuit diagram of an Arduino based braille display for the visually impaired describes the detailed summary about the components used , and the way of interfacing between the components.

The entire project setup is clearly shown in Fig 3.14

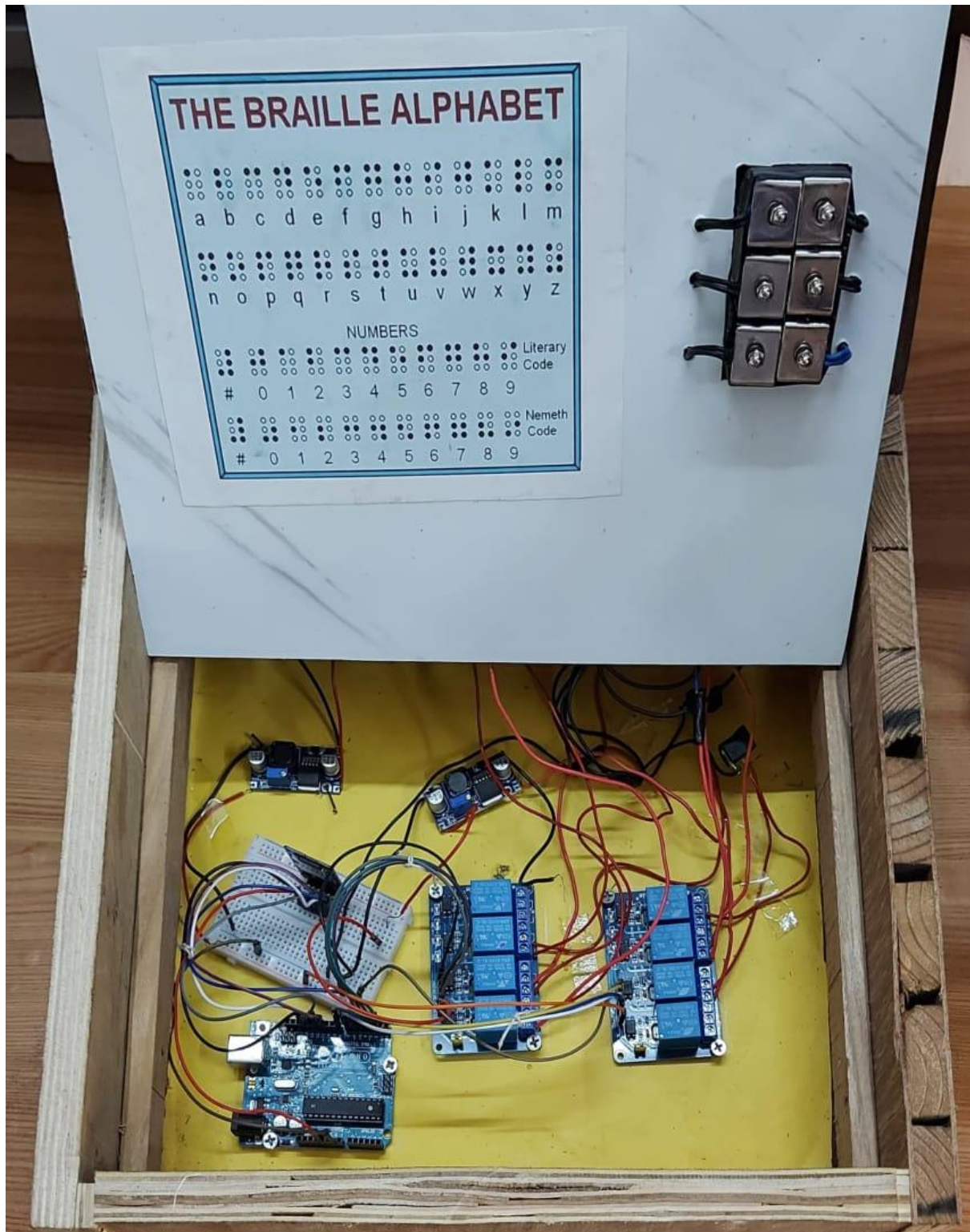


Fig 3.14: Project setup of an Arduino based braille display for the visually impaired

3.4. LOOK-UP TABLE

The following Table 3.2 consists of list of all letters & numbers associated in braille representation format via numbering of solenoids to be active HIGH in the implemented solenoid display.

Character	Solenoids to be HIGH	Character	Solenoids to be HIGH
A/a	1	N/n	1,3,4,5
B/b	1,2	O/o	1,3,5
C/c	1,4	P/p	1,2,3,4
D/d	1,4,5	Q/q	1,2,3,4,5
E/e	1,5	R/r	1,2,3,5
F/f	1,2,4	S/s	2,3,4
G/g	1,2,4,5	T/t	2,3,4,5
H/h	1,2,5	U/u	1,3,6
I/i	2,4	V/v	1,2,3,6
J/j	2,4,5	W/w	2,4,5,6
K/k	1,3	X/x	1,3,4,6
L/l	1,2,3	Y/y	1,3,4,5,6
M/m	1,3,4	Z/z	1,3,5,6

Table 3.2- Braille mapping of letters into solenoid display

Character	Solenoids to be HIGH	Character	Solenoids to be HIGH
1	2	8	2,3,6
2	2,3	9	3,5
3	2,5	0	3,5,6
4	2,5,6	#	3,4,5,6
5	2,6	,	2,5,6
6	2,3,5	.	2
7	2,3,5,6		

Table 3.3- Braille mapping of all other characters into solenoid display

Chapter 4: IMPLEMENTATION

This chapter describes the flowchart, design and a block-by-block implementation of an Arduino based braille display for the visually impaired project.

4.1. Flowchart

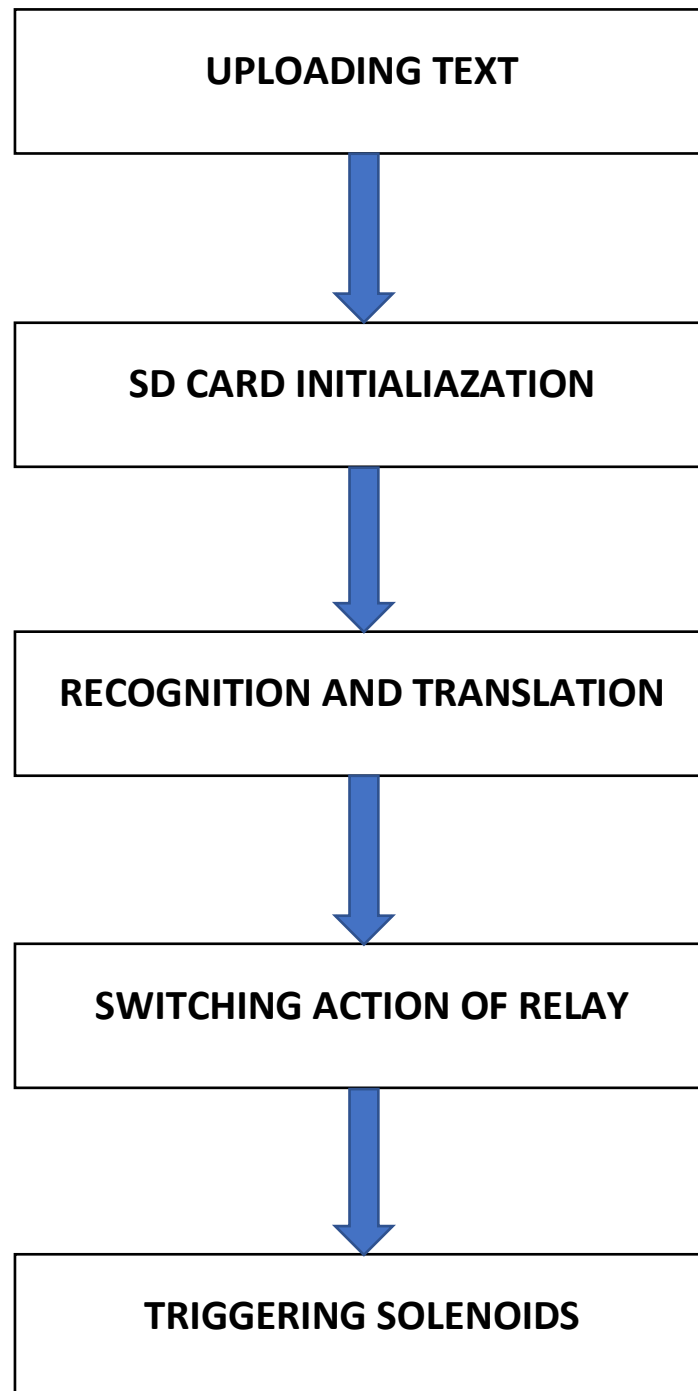


Fig 4.1- Flowchart of an Arduino based braille display for the visually impaired

4.2. UPLOADING TEXT

The steps involved in uploading text into the SD Card is as follows:

- In this initial step, the text which should be implemented is saved as a file in '.txt' format since the Arduino UNO can only read '.txt' files.
- This file is then uploaded into SD card via any data transfer device like a simple card reader.
- This SD card is then inserted into the SD card module.
- Here the SD card should be formatted as FAT16 or FAT32 as here SD card module is connected with Arduino.

The pictorial representation of uploading text is as follows:

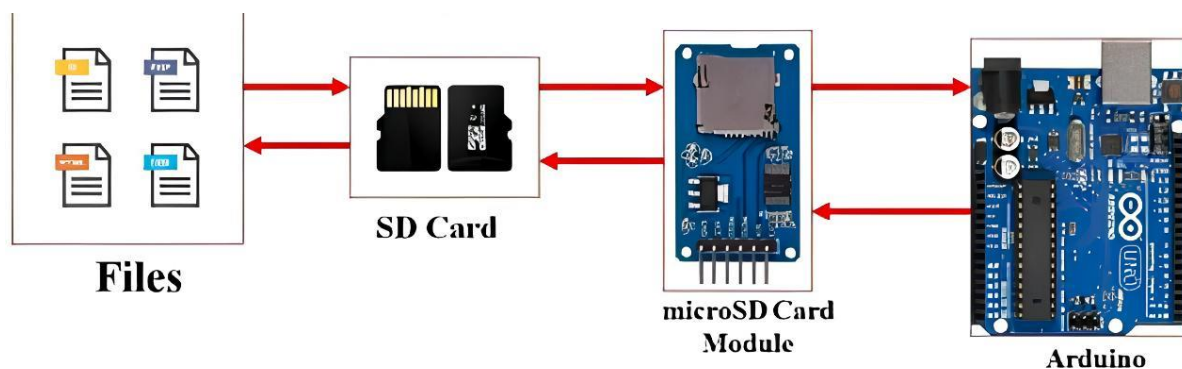


Fig 4.2: Uploading text files into an SD card

4.3. SD CARD INITIALIZATION

Here the SD card module and Arduino are connected by SPI pins [12]. To enable them 'SPI.h' header file is used. The SPI bus is a synchronous serial communication interface that allows devices to communicate with each other over short distances. The functions provide by the 'SPI.h' library allows the Arduino to send commands to the SD card and manage the flow of data between the two devices.

SPI uses the following four wires –

- **SCK** – This is the serial clock driven by the master.
- **MOSI** – This is the master output / slave input driven by the master.
- **MISO** – This is the master input / slave output driven by the master.
- **CS** – This is the chip-selection wire.

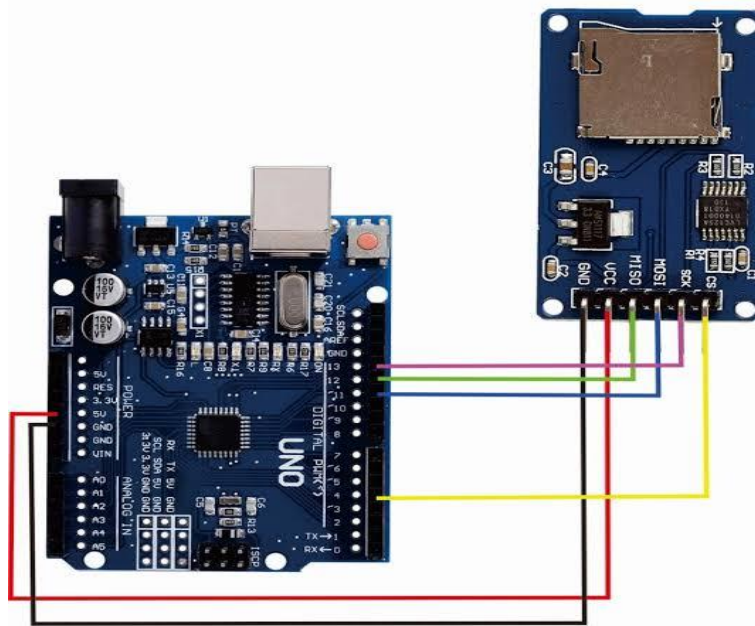


Fig 4.3: SD card initialization

Important points to note while initialization of SD card:

- Here the connection between Arduino and the computer is established using the command 'serial.begin(9600)'.
- It is used to output status messages that informs the user if the initialization of the SD card has succeeded or failed. Here 9600 refer to the baud rate.
- The baud rate is the rate at which data is transmitted over a serial connection. It is a measure of the number of signal changes that occur per second. It means that the data is being transmitted at a rate of 9600 bits per second between the Arduino and the computer connected to it.
- 9600 is a commonly used baud rate because it is a standard rate that is widely supported and it is good balance between speed and reliability.
- Here the chip select wire place a major role for initialization of SD card. If CS pin is connected as defined in the code, then initialization of the SD card is done.

4.4. RECOGNITION AND TRANSLATION

After initialisation of the SD card is done, 'SD.h' header file is used to access SD card information. It provides functions for working with SD cards on Arduino boards. It allows the Arduino to initialize the SD card, open and close files on the card, read and write data to and from the card. And perform other operations related to SD card management. If the file provided in code is available in the SD card, then the corresponding file is opened. The Arduino reads one character at a time. The text in the file can be printed in the serial monitor.

After the recognition of text in the SD card, each letter in the text is translated into braille language by a user defined function 'DisplayBraille()'. The function contains a block of code that converts the braille code to its corresponding braille representation using conditional statements and string manipulation.

4.5. SWITCHING ACTION OF RELAY

A relay module is an electrical switch that is operated by an electromagnet. The electromagnet is activated by a separate low-power signal from a micro controller which is Arduino. When activated, the electromagnet pulls to either open or close an electrical circuit namely relay performs switching action [13]. This project uses 2 identical 5V 4-Channel relays in order to functionally drive the solenoids.

In general, these are the steps involved in switching action of relay:

- The input of relay is given from the digital pins of Arduino which sends signal.
- 5V is supplied from buck circuit to power up the relay & proper grounding is done.
- "DigitalWrite()" in the IDE coding allows a particular pin to go to HIGH state, in fact allowing it to perform proper switching of relay.
- The signal is sent in such a way that for a particular character the specified braille code is sent to the 6 inputs of relay whether they are high or low. When the relay is active then it is high and a red led light will glow.
- In this project, usually load is connected and operated at NO terminal end, so that whenever the DigitalWrite goes HIGH, the switching takes place from NC to NO end.

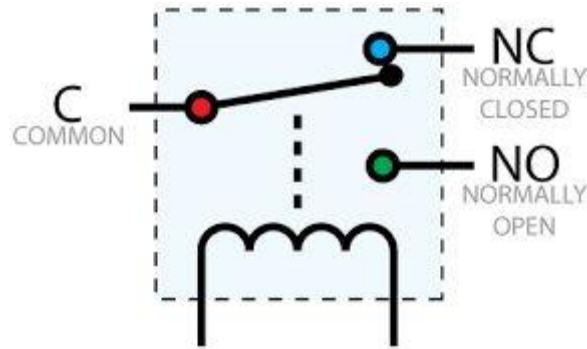


Fig 4.4: Relay switching mechanism

4.6. TRIGGERING SOLENOIDS

As mentioned before, the push-pull solenoids are connected to NO terminal of relay module. These solenoids should be connected to corresponding channel as per the order defined in the code. The positive terminal of 12V power supply for the solenoids to drive is given to the common terminals of each channel of relay and also all the ground terminals of solenoids are commonly grounded along with negative terminal of 12V power supply and ground pins of both relay modules and Arduino. Based on the switching action of the relay module corresponding solenoids are triggered to create braille impression for each character.



Fig 4.5: Braille cell using solenoids triggering

Based on the inputs given to the relay by the Arduino digital pins, the relay starts switching on the respective solenoids connected to the NO terminal.

The 12V supply provides the necessary & sufficient power to drive the solenoids to HIGH state. The delay upto which the solenoids need to be high will be decided by the written code.

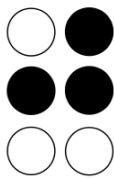
Typically, the solenoids are in HIGH state as long as the “DigitalWrite()” for the respective digital pin is HIGH.

This chapter concludes about the implementation of an Arduino based braille display for visually impaired project, also a flowchart and a block-by-block detailed explanation of working procedure involved.

Chapter 5: RESULTS AND DISCUSSION

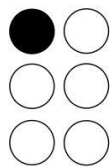
5.1. EXAMPLE OUTPUT 1:

This chapter typically summarizes the findings and outcomes of this project. This includes the data that was given as input and the analysis performed while doing this project.



J

www.TrackAndField.com



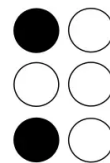
A

www.TrackAndField.com



C

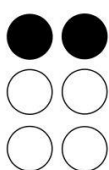
www.TrackAndField.com



K

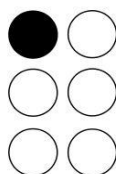
www.TrackAndField.com

5.2. EXAMPLE OUTPUT 2:



C

www.TrackAndField.com



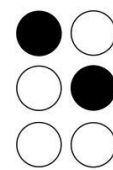
A

www.TrackAndField.com



F

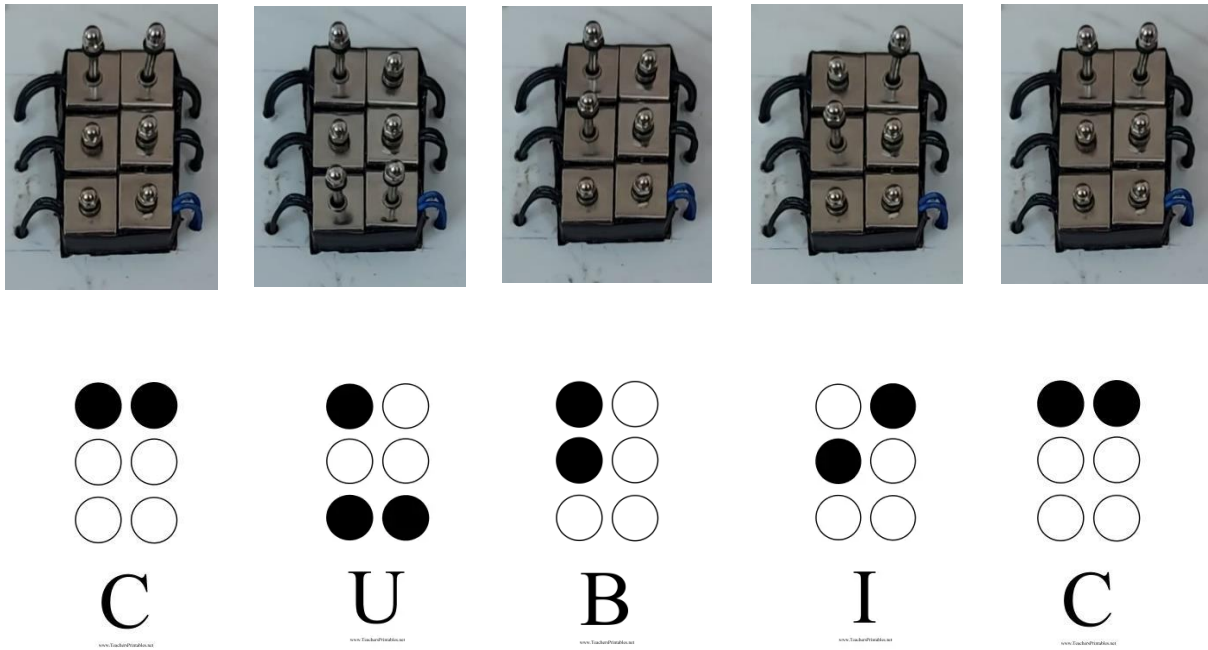
www.TrackAndField.com



E

www.TrackAndField.com

5.3. EXAMPLE OUTPUT 3:



5.4. EXAMPLE OUTPUT 4:

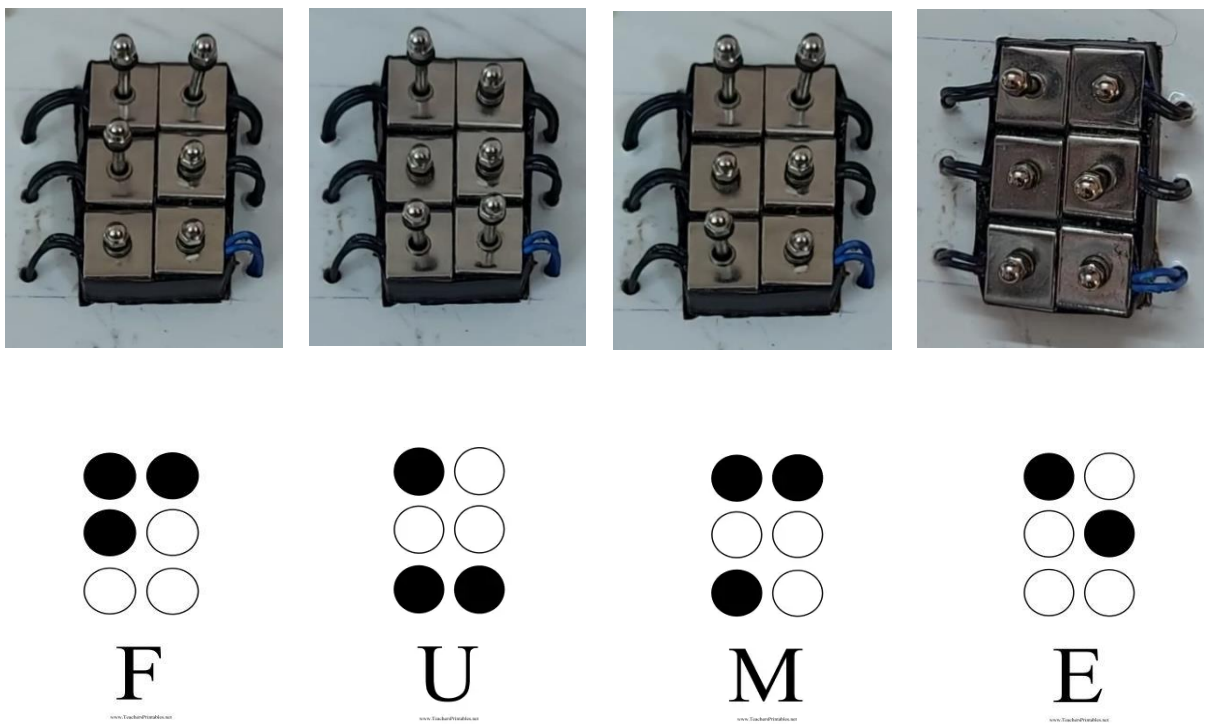


Fig 5.1: Outputs of an Arduino based braille display for the visually impaired

5.5. Applications

- **Accessibility:** The converter provides a means for people with visual impairments to access printed materials.
- **Convenience:** The converter is a portable and easy-to-use device that can be used on the go.
- **Cost-effective:** Braille printers can be expensive, but a text to braille converter can be an affordable alternative.
- **Time-saving:** The converter can quickly translate text to braille, which saves time compared to manually transcribing documents.
- **Learning aid:** The converter can be used as a learning aid for those who are learning braille.

5.6. Advantages and Disadvantages

Some potential advantages of this project could include:

- **Increased accessibility:** The project could help make printed or digital text more accessible for visually impaired individuals who use braille as their primary reading and writing system.
- **Cost-effective:** Creating a text to braille converter using open-source software and off-the-shelf hardware components can be a cost-effective solution compared to commercial Braille devices.
- **Customization:** With a text to braille converter project, users can customize the settings and output to meet their individual needs and preferences.

Some potential disadvantages of this project could include:

- **Limitations in accuracy:** The accuracy of the braille output may not be perfect due to variations in the braille translation algorithms or potential errors in the conversion process.
- **Compatibility issues:** The project may not be compatible with all types of digital or printed text, and users may need to convert files into a specific format before using the converter.

CONCLUSION AND FUTURE SCOPE

Conclusion

This project demonstrates the potential for low-cost and accessible assistive technology devices that can improve the lives of individuals with visual impairments. By using an Arduino microcontroller, which is responsible for running a translation algorithm to convert stored text into braille script.

The Arduino is interfaced to SD card module to access the text and also interfaced to relays to perform switching action. The push-pull solenoids are then interfaced with relay module in order to create a refreshable braille impression.

Now a days as technology is increasing, from books to smartphones (i.e., Kindle, Tabs, etc). But for the case of visually impaired, it should be from braille books to refreshable braille displays, that are designed in this project.

Future Scope

To make it more user-friendly, this device can be further improved by introducing the following features.

- Adding speech output to the device would allow visually impaired users to hear the text being converted to braille. This would make the device more accessible to users who may have difficulty reading braille.
- Incorporating a microphone would allow users to input text using their voice instead of typing it in. This would be particularly useful for users with limited hand mobility.
- Adding Bluetooth connectivity would allow users to connect their smartphones to the device and input text using their mobile devices.
- A language translation software can be incorporated into the device to automatically translate text into English and then into braille for output.

BIBLIOGRAPHY

- [1] Ghose R, *et.al.*, "Architecture of a web browser for visually handicapped people", Students' Technology Symposium (TechSym), IEEE, pp. 325- 329, 3-4 April 2010.
- [2] P. H. Zope, *et.al.*, "Design and Implementation of Messaging System Using Braille Code for Virtually Impaired Persons", International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering (An ISO 3297: 2007 Certified Organization) Vol. 5, Issue 7, July 2016.
- [3] P. J. Swati Malik, published paper titled "Low-Cost Portable E-Braille Device for Blind and Visually Impaired Persons", International Journal of innovation Research Computers Communication Engineering, Volume No. 3297, Issue No. 6, pp. 11449–11455, 2016.
- [4] Priyanka D. Joshi, *et.al.*, "Microcontroller Based Smart Touch Gadget Assistant for Visually Impaired", in International Journal of Advanced Research in Engineering, Science & Technology (IJAREST) Volume 5, Issue 6, June 2018.
- [5] S. B. Sibila.R, *et.al.*, published paper titled "Text to Braille Conversion Using MATLAB", in International Journal of Science Research Innovation, Volume No. 11, Issue No. 1, pp. 1–8, 2018.
- [6] S. Bharath Reddy, *et.al.*, "Design of a Tactile Braille Cell", in 7th IEEE International Conference on Advances in Computing, Communications, and Informatics (ICACCI), PES Institute of Technology, 2018.
- [7] A V Hemanth, *et.al.*, "Arduino and Solenoid Valve-Based Text to Braille Converter", Proceedings of the Third International Conference on Smart Systems and Inventive Technology, 2020.
- [8] Rhea Sawant, *et.al.*, "Text to Braille Conversion System", International Conference on Innovative Computing, Intelligent Communication and Smart Electrical Systems(ICSES), 2021.
- [9] G.S.N. RAJU, "Electro Magnetic Field theory and Transmission Lines", Andhra Pradesh, India: Pearson Education India 2006.
- [10] Ashish Bansal "Arduino Course for Beginners - Open-Source Electronics Platform".
<https://www.freecodecamp.org/news/create-your-own-electronics-with-arduino-full-course/>.
- [11] "Interfacing 5V 4-Channel Relay module with Arduino UNO"
<https://microcontrollerslab.com/relay-module-interfacing-arduino-arduino/>

[12] “Interfacing SD card module with Arduino UNO”.

<https://lastminuteengineers.com/arduino-micro-sd-card-module-tutorial>

[13] “Interfacing 12V Solenoids with 5V 4-Channel Relay module”.

<https://techatronic.com/solenoid-valve-interfacing-with-arduino/>

APPENDIX

CODE

```
#include <SPI.h>
#include <SD.h>

const int chipSelect = 4;
const int s1 = 2;
const int s2 = 3;
const int s3 = 5;
const int s4 = 6;
const int s5 = 7;
const int s6 = 8;

void setup() {
    Serial.begin(9600);

    if (!SD.begin(chipSelect)) {
        Serial.println("SD card initialization failed.");
        return;
    }
    Serial.println("SD card initialization done.");
    pinMode(s1, OUTPUT);
    pinMode(s2, OUTPUT);
    pinMode(s3, OUTPUT);
    pinMode(s4, OUTPUT);
    pinMode(s5, OUTPUT);
```

```

    pinMode(s6, OUTPUT);
}

void loop() {
    File file = SD.open("tests.txt");

    if (file) {
        while (file.available()) {
            char letter = file.read();
            displayBraille(letter);
            delay(490);
            resetSolenoids();
            delay(490);
        }
        file.close();
    } else {
        Serial.println("Error opening file.");
    }
}

void displayBraille(char letter) {
    switch (letter) {
        case 'a':
        case 'A':
            digitalWrite(s1, LOW);
            digitalWrite(s2, HIGH);
            digitalWrite(s3, HIGH);
            digitalWrite(s4, HIGH);
            digitalWrite(s5, HIGH);

```

```

        digitalWrite(s6, HIGH);
        break;

case 'b':
case 'B':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'c':
case 'C':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, LOW);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'd':
case 'D':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, LOW);

```

```

        digitalWrite(s5, LOW);
        digitalWrite(s6, HIGH);
        break;

case 'e':
case 'E':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, HIGH);
    break;

case 'f':
case 'F':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, LOW);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'g':
case 'G':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);

```

```

        digitalWrite(s4, LOW);
        digitalWrite(s5, LOW);
        digitalWrite(s6, HIGH);
        break;

case 'h':
case 'H':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, HIGH);
    break;

case 'i':
case 'I':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, LOW);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'j':
case 'J':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);

```

```

        digitalWrite(s3, HIGH);
        digitalWrite(s4, LOW);
        digitalWrite(s5, LOW);
        digitalWrite(s6, HIGH);
        break;

case 'k':
case 'K':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'l':
case 'L':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;

case 'm':
case 'M':
    digitalWrite(s1, LOW);

```

```
digitalwrite(s2, HIGH);  
digitalwrite(s3, LOW);  
digitalwrite(s4, LOW);  
digitalwrite(s5, HIGH);  
digitalwrite(s6, HIGH);  
break;
```

```
case 'n':
```

```
case 'N':
```

```
digitalwrite(s1, LOW);  
digitalwrite(s2, HIGH);  
digitalwrite(s3, LOW);  
digitalwrite(s4, LOW);  
digitalwrite(s5, LOW);  
digitalwrite(s6, HIGH);  
break;
```

```
case 'o':
```

```
case 'O':
```

```
digitalwrite(s1, LOW);  
digitalwrite(s2, HIGH);  
digitalwrite(s3, LOW);  
digitalwrite(s4, HIGH);  
digitalwrite(s5, LOW);  
digitalwrite(s6, HIGH);  
break;
```

```
case 'p':
```

```
case 'P':
```

```
digitalwrite(s1, LOW);  
digitalwrite(s2, LOW);  
digitalwrite(s3, LOW);  
digitalwrite(s4, LOW);  
digitalwrite(s5, HIGH);  
digitalwrite(s6, HIGH);  
break;
```

```
case 'q':
```

```
case 'Q':
```

```
digitalwrite(s1, LOW);  
digitalwrite(s2, LOW);  
digitalwrite(s3, LOW);  
digitalwrite(s4, LOW);  
digitalwrite(s5, LOW);  
digitalwrite(s6, HIGH);  
break;
```

```
case 'r':
```

```
case 'R':
```

```
digitalwrite(s1, LOW);  
digitalwrite(s2, LOW);  
digitalwrite(s3, LOW);  
digitalwrite(s4, HIGH);  
digitalwrite(s5, LOW);  
digitalwrite(s6, HIGH);  
break;
```

```
case 's':
```

```
case 's':  
    digitalWrite(s1, HIGH);  
    digitalWrite(s2, LOW);  
    digitalWrite(s3, LOW);  
    digitalWrite(s4, LOW);  
    digitalWrite(s5, HIGH);  
    digitalWrite(s6, HIGH);  
    break;
```

```
case 't':
```

```
case 'T':  
    digitalWrite(s1, HIGH);  
    digitalWrite(s2, LOW);  
    digitalWrite(s3, LOW);  
    digitalWrite(s4, LOW);  
    digitalWrite(s5, LOW);  
    digitalWrite(s6, HIGH);  
    break;
```

```
case 'u':
```

```
case 'U':  
    digitalWrite(s1, LOW);  
    digitalWrite(s2, HIGH);  
    digitalWrite(s3, LOW);  
    digitalWrite(s4, HIGH);  
    digitalWrite(s5, HIGH);  
    digitalWrite(s6, LOW);  
    break;
```

```

case 'v':
case 'V':
    digitalWrite(s1, LOW);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, LOW);
    break;

case 'w':
case 'W':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, LOW);
    digitalWrite(s5, LOW);
    digitalWrite(s6, LOW);
    break;

case 'x':
case 'X':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, LOW);
    digitalWrite(s4, LOW);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, LOW);
    break;

```

```
case 'y':
case 'Y':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, LOW);
    digitalWrite(s4, LOW);
    digitalWrite(s5, LOW);
    digitalWrite(s6, LOW);
    break;

case 'z':
case 'Z':
    digitalWrite(s1, LOW);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, LOW);
    break;

case ',':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, LOW);
    break;
```

```
case ' ':
```

```
    digitalWrite(s1, HIGH);  
    digitalWrite(s2, HIGH);  
    digitalWrite(s3, HIGH);  
    digitalWrite(s4, HIGH);  
    digitalWrite(s5, HIGH);  
    digitalWrite(s6, HIGH);  
    break;
```

```
case '.':
```

```
    digitalWrite(s1, HIGH);  
    digitalWrite(s2, LOW);  
    digitalWrite(s3, HIGH);  
    digitalWrite(s4, HIGH);  
    digitalWrite(s5, HIGH);  
    digitalWrite(s6, HIGH);  
    break;
```

```
case '#':
```

```
    digitalWrite(s1, HIGH);  
    digitalWrite(s2, HIGH);  
    digitalWrite(s3, LOW);  
    digitalWrite(s4, LOW);  
    digitalWrite(s5, LOW);  
    digitalWrite(s6, LOW);  
    break;
```

```
case '0':
```

```
    digitalWrite(s1, HIGH);
```

```

        digitalWrite(s2, HIGH);
        digitalWrite(s3, LOW);
        digitalWrite(s4, HIGH);
        digitalWrite(s5, LOW);
        digitalWrite(s6, LOW);
        break;
case '1':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;
case '2':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, HIGH);
    break;
case '3':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, HIGH);

```

```

        break;
case '4':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, LOW);
    break;
case '5':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, HIGH);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, LOW);
    break;
case '6':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, HIGH);
    break;
case '7':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);

```

```

        digitalWrite(s4, HIGH);
        digitalWrite(s5, LOW);
        digitalWrite(s6, LOW);
        break;
case '8':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, LOW);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, HIGH);
    digitalWrite(s6, LOW);
    break;
case '9':
    digitalWrite(s1, HIGH);
    digitalWrite(s2, HIGH);
    digitalWrite(s3, LOW);
    digitalWrite(s4, HIGH);
    digitalWrite(s5, LOW);
    digitalWrite(s6, HIGH);
    break;

default:
    break;
}
}

void resetSolenoids() {
    digitalWrite(s1, HIGH);
    digitalWrite(s2, HIGH);

```

```
digitalwrite(s3, HIGH);  
digitalwrite(s4, HIGH);  
digitalwrite(s5, HIGH);  
digitalwrite(s6, HIGH);  
}
```